The Rule Conflict Resolution Boundary in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 7, 2026

—

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one boundary case for the architecture — the rule conflict resolution boundary, in which two orchestration rules authored under the source paper's human-governance commitment specify different cell behaviors for the same substrate state — and to articulate the architectural treatment that follows from already-defended commitments without introducing automatic precedence as a new architectural device.

Abstract

The CKS pattern's conflict-as-first-class commitment (§5 of the source paper) addresses contradictions in object-level substrate content. A second class of contradiction, distinct in scope but architecturally adjacent, arises in the rule frame itself: two human-authored orchestration rules that specify different cell behaviors for the same substrate state. This is the **rule conflict resolution boundary** — a stress test of the conflict-as-first-class commitment under extension to the rules that govern cell behavior. Many systems treat such conflicts through automatic precedence (newer-wins, more-specific-wins, rule-priority-field, vendor-default-precedence). CKS rejects all such automatic resolution as instances of the contradiction-collapse-by-automation anti-pattern. The architectural treatment is instead: register the rule conflict as substrate-resident first-class conflict, attach provenance distinguishing the conflicting rules, accommodate the conflict period without forced resolution, and resolve through human authoring of an explicit meta-rule. This note formalizes the boundary, identifies the architectural commitments stressed, articulates the treatment, names the anti-pattern treatments that violate it, and bounds its scope.

1. Why the rule conflict resolution boundary needs standalone formalization

The CKS pattern's conflict-as-first-class commitment (§5 of the source paper) commits the substrate to preserving contradictions as first-class addressable objects with their own identity and provenance. The commitment is developed for object-level conflicts — two facts about the same entity, two decisions in tension — with both substrate-level preservation and cell-level resolution under orchestration rules specified for that case.

The commitment also reaches a second case the source paper does not develop in its own subsection: conflict between two orchestration rules, both authored under human authority, both substrate-resident, that specify different cell behaviors for the same substrate state. This case is architecturally adjacent to object-level conflict but operationally distinct — it reaches not the object-level content cells coordinate over but the rule frame the cells execute under. Under stress, the case has a non-obvious failure mode: deployments reach for an automatic precedence rule (newer-wins, specific-over-general, priority-field-over-rule, vendor-default-precedence) and apply it without registering the conflict or surfacing it for human resolution. Each such move is an instance of the contradiction-collapse-by-automation anti-pattern, the same anti-pattern §5 names in the object-level case but operating one frame higher.

The standalone formalization matters because the surrounding literature on policy management, rule engines, and compliance frameworks defines automatic-precedence policies as the standard treatment for rule conflicts. If the CKS architecture's response is left implicit, deployments default to whatever precedence policy their rule engine, policy framework, or compliance vendor supplies — almost always an automatic-precedence policy. The architectural treatment must be stated explicitly to be defensible against the slide.

This note opens Phase A6 of the derivation series, which formalizes approximately fifteen boundary cases in which standard commitments produce non-obvious outcomes. The rule conflict resolution boundary is the opening case because it is the first boundary practitioners encounter in any production deployment with more than a handful of rules.

2. The boundary case scenario

Two rules, both authored under the source paper's human-governance commitment, both substrate-resident, both authoritative, conflict on the same substrate state input. The conflict is structural: the rules specify different cell behaviors for the same input. They may have arisen through any of the typical paths — drafted at different times by the same human without the earlier rule in view, drafted by different humans with overlapping responsibility scopes, drafted in different parts of the substrate by humans in different roles, or drafted as default and exception-handler in a regime where the boundary between regimes is not crisp at runtime.

What makes the case non-obvious is that there is no architectural ground for preferring one rule over the other. Both are authoritative, substrate-resident, and human-authored. The conflict is not an artifact of one rule being a draft, a mistake, or an outdated version — those cases are addressed by the inspect and modify rights and do not reach the boundary at issue. The conflict is an artifact of two legitimate rules covering the same state. A cell encountering substrate state covered by both cannot proceed without selecting between them. The architectural question is what selection mechanism is admissible.

3. Which architectural commitments are stressed

Four commitments are stressed by the boundary, and each clarifies what the treatment must preserve.

Conflict-as-first-class is stressed by extension to the rule frame. The source paper's §5 develops the commitment for object-level contradictions. Rule conflict is a contradiction in the rule frame: not "the

substrate disagrees about an entity" but "the substrate's rules disagree about how to act on an entity." If rules can be silently merged, discarded, or auto-resolved, the architectural property §5 defends in object-level content is being violated one frame up.

Human-governance is stressed at the meta-rule layer. The source paper's commitment to human authority over orchestration rule authoring (§3.3) means that the resolution of rule conflicts must itself be human-authored. There is no privileged third party — no vendor, no LLM, no framework — that can resolve a conflict between two human-authored rules without itself becoming an authoring actor outside human authority.

Substrate accommodation of conflicting state is stressed at the rule frame. The substrate must accommodate two conflicting rules without forcing one out or marking either as inactive without human authority — the analogue, at the rule frame, of object-level conflict accommodation.

Substrate as source of truth for "what rules apply" is stressed because the question becomes momentarily ambiguous. Before a meta-rule is authored, the question "which rule governs this state" has no unique answer. The substrate is the source of truth for what rules the substrate carries (§11.3); what the substrate carries is *both* of them. The treatment cannot resolve the ambiguity by removing one rule without human action; it must accommodate the ambiguity until human authority resolves it.

4. The architectural treatment

The treatment is the direct extension of the conflict-as-first-class commitment to the rule frame, with no additional architectural devices.

Register the rule conflict as substrate-resident first-class conflict. When a cell or governance actor detects that two rules specify different behaviors for the same substrate state input, the conflict is recorded in the substrate as a first-class object — addressable, traceable, with its own identity. The conflict object references both rules and is substrate state, not transient runtime state; it persists until a meta-rule resolves it or a human modifies one of the conflicting rules.

Attach provenance distinguishing the conflicting rules. The conflict object carries the provenance the architecture commits to for any first-class conflict: writer, timestamp, rationale, relationship to the conflicting content, governance authority under which each rule was authored, and the cell or governance actor that first exposed the conflict. The provenance distinguishes the two rules — who authored each, when, under what authority, in response to what conditions — making the conflict auditable and giving the human authoring the meta-rule the information needed to resolve it.

Apply no automatic precedence. The architecture explicitly rejects newer-wins, specific-over-general, broader-scope-wins, rule-priority-field-precedence, vendor-default-precedence, and any other automatic resolution rule. The rejection is architectural, not preferential. Every such move is an instance of the contradiction-collapse-by-automation anti-pattern, the same anti-pattern §5 names for object-level conflicts.

Resolve through human authoring of a meta-rule. The resolution mechanism is a rule, authored by a human under the same human-governance commitment as the original two, that specifies which of the two conflicting rules applies in the conflicting condition. The meta-rule is itself substrate-resident, itself

first-class content, itself authoritative. It does not delete the conflicting rules or merge them; it specifies a selection between them under the conflicting condition. Both original rules persist after the meta-rule is authored, and other cells encountering them under non-conflicting conditions continue to apply them as before.

Allow recursive meta-rule structure. A meta-rule may itself conflict with another meta-rule, requiring a meta-meta-rule to resolve. The recursion is not architecturally problematic. Each level is governed by humans through the source paper's authority commitment, substrate-resident through the substrate's accommodation of conflicting state, and registered through the conflict-as-first-class commitment. The same treatment applies at every level; there is no level at which the architecture's commitments cease to hold.

Accommodate the conflict period without forced resolution. Between the moment the rule conflict is registered and the moment a meta-rule resolves it, cells encountering substrate state covered by both rules may produce different outcomes depending on which rule they consult. This is operationally acceptable because the conflict is registered (governance-visible), the provenance distinguishes which rule each cell used (audit preserved), and any decision premised on the unresolved conflict can be revisited under the meta-rule once authored. Path retraceability survives the conflict period.

The treatment introduces no new architectural devices. Every move is the direct application of commitments the source paper already defends.

5. Anti-pattern treatments that violate the architecture

Five named treatments, each common in surrounding policy management and rule engine literature, instantiate the contradiction-collapse-by-automation anti-pattern when applied to rule conflicts.

Automatic precedence (newer-wins, more-specific-wins, broader-scope-wins). A rule engine resolves the conflict at execution time by applying a precedence policy without registering the conflict, attaching provenance, or surfacing the conflict for human meta-rule authoring. The rule that loses precedence is silently set aside. This is the canonical instance of contradiction-collapse-by-automation at the rule frame.

Vendor-managed rule precedence. A vendor-supplied rule framework defines precedence rules in its product configuration, outside the substrate. Cells defer to the vendor's policy. The conflict is resolved by an actor outside the substrate's human-governance scope and outside the substrate's accommodation of conflicting state, violating both human-governance and substrate-as-source-of-truth: the answer to "which rule applies" cannot be read from substrate content alone.

LLM-mediated rule resolution. A cell, encountering two conflicting rules, calls an LLM and asks it to determine which rule applies. The LLM's selection is then treated as the operative rule. The treatment relocates resolution authority to the LLM, in violation of the AI-as-substrate-mediator commitment that locates authority in human-authored substrate content rather than in the LLM. It is also an instance of contradiction-collapse-by-automation, with the LLM as the collapsing actor.

Framework-default precedence. A deployment framework — a workflow engine, an agent framework, a policy DSL — imposes a default precedence ordering as a system feature. Cells inherit

the default unless explicitly overridden. The treatment makes precedence a property of the deployment frame rather than of the substrate, again violating substrate-as-source-of-truth.

Rule-priority-field patterns. Rules are authored with a priority metadata field, and the engine resolves conflicts by comparing priority values. The priority field functions as automatic precedence: as soon as two rules with different values conflict, the engine resolves without registering the conflict. The architectural failure is not the priority field as an annotation — annotations on rules are admissible substrate content — but the engine's use of the field as a resolution rule. Even when priority values are human-authored, treating priority comparison as the resolution mechanism is contradiction-collapse-by-automation. A priority annotation that informs human meta-rule authoring is admissible; a priority field the engine uses to auto-select is not.

These five names cover the great majority of automatic-precedence treatments encountered in production rule and policy engines. Naming them as anti-patterns explicitly is what makes the architectural treatment defensible against the slide the surrounding literature otherwise produces by default.

6. Operational implications

Cells during the conflict period may produce different outcomes for the same substrate state. This is the direct consequence of accommodating the conflict rather than auto-resolving it. The substrate-layer deterministic-state guarantee is preserved (substrate content does not change as a function of the conflict), but cell-level behavior is undetermined for inputs covered by the conflict until the meta-rule resolves it. The architecture is committed to this cost — undetermined cell behavior visibly registered as a known unresolved conflict — rather than to the alternative cost: silent loss of governance authority over rule selection.

The conflict is governance-visible, and audit identifies all cell executions that exposed it. Because the conflict is registered as substrate content, humans exercising the inspect right see that a meta-rule is needed; governance does not depend on a runtime alert, a reviewer workflow, or an LLM-generated summary. Because each cell execution records in its own provenance which rule it consulted, the audit trail can identify every execution that ran under either rule during the conflict period. A meta-rule authored after the fact can be applied retroactively to those executions or prospectively only; the architecture supports both, and the choice is a deployment matter.

Meta-rule authoring is the resolution mechanism, not auto-detection or LLM summarization. Tooling can detect rule conflicts, surface them, summarize them, and recommend meta-rules for human consideration. What is not permissible is for the tooling to author the meta-rule. The meta-rule must be authored by a human exercising the source paper's authoring authority, regardless of what tooling assists in drafting it.

7. Limits of the architectural treatment

The treatment applies specifically to conflicts within the rule frame. Three classes of conflict are out of scope and are addressed elsewhere.

Cell-internal consultation conflicts. When a cell consults two sources during execution and the sources return inconsistent values for the same input — an LLM consultation versus a rule output, two LLM consultations differing in result, a substrate read versus an LLM-derived projection — the conflict is at the cell-internal consultation frame, not the rule frame. This is addressed by the conflict-as-first-class commitment as it operates within cells.

Authority distribution conflicts. When two humans claim authoring authority over the same rule under different role definitions, the conflict is at the authority distribution frame — who can author this rule — rather than at the content of the rules themselves. Authority distribution conflicts are formalized in a separate Phase A6 boundary case.

Composition-level rule conflicts. When two CKS substrates are composed and rules from each conflict over coordinated content at the composition boundary, the conflict involves composition requirements that the source paper develops in §13.3. Composition-level conflicts are governed by the architecture's composition commitments, not by the within-substrate rule conflict treatment specified here.

These narrowings are deliberate. The treatment in §4 covers the within-substrate rule conflict case precisely; extending it to the three out-of-scope cases would import architectural commitments those cases require but the treatment here does not supply.

8. Operational test

A deployment treats the rule conflict resolution boundary in CKS-coherent fashion if and only if all of the following are true at all times during the substrate's existence:

1. Rule conflicts, when detected, are registered as first-class substrate-resident conflict objects with provenance distinguishing the conflicting rules.
2. No precedence policy — newer-wins, specific-over-general, broader-scope-wins, rule-priority-field, vendor-default-precedence, framework-default-precedence — is applied to resolve the conflict at execution time without human-authored meta-rule.
3. Resolution of rule conflicts proceeds exclusively through human authoring of meta-rules under the source paper's human-governance commitment.
4. Meta-rules that themselves conflict are treated under the same architectural treatment recursively, with no level at which automatic precedence is admissible.
5. Cells executing under unresolved rule conflicts record in provenance which rule each execution consulted, preserving path retraceability through the conflict period.

A deployment that fails any of (1)–(5) is treating rule conflicts in a manner not CKS-coherent, even if it satisfies the conflict-as-first-class commitment for object-level content. The boundary is a separate axis of compliance.

9. Conclusion

The rule conflict resolution boundary stress-tests the conflict-as-first-class commitment by extending it to the rule frame. Without standalone formalization, deployments default to automatic-precedence

treatments the surrounding policy management literature supplies, all of which instantiate the contradiction-collapse-by-automation anti-pattern. The architectural treatment is the direct extension of already-defended commitments: register the conflict, attach provenance, reject automatic precedence, resolve through human-authored meta-rule, accommodate recursion through the same treatment at every level. No new architectural devices are introduced.

This note opens Phase A6 of the derivation series. Subsequent Phase A6 notes formalize boundary cases in rule retroactivity, vendor unavailability, LLM consultation timeout, composition partner failure, and approximately ten further scenarios; together the phase concludes Series A's coverage of the source paper's architectural derivations.

Subsequent work that handles rule conflicts in CKS-instantiated systems should use the rule conflict resolution boundary in the sense formalized here. Subsequent work that uses automatic precedence to resolve rule conflicts is not CKS-coherent on this axis, regardless of how robustly it satisfies other commitments, and the difference should be named.

—

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Rule Conflict Resolution Boundary in the Coordination Knowledge Substrate Pattern.* May 7, 2026. ORCID: 0009-0004-8065-3235.